Bottom Busters

Game Design Document, first draft (short form)

Multi-Agent AI for Game Development, ELVTR. Assignment #01. David, Tingle Games LLC (Austin, TX). October 2026. Engine: Unity 6.3 LTS. Platform: Windows on Steam and itch.io.

1. Executive Summary

Bottom Busters is a 2D arcade action platformer for one or two players on one screen, inspired by Nintendo's Balloon Fight. Every character floats like a balloon and flies by farting. Each character has two cheeks for health. You land on an enemy from above to pop its cheeks; lose both of yours and you deflate and fly off the screen. Water kills instantly. Thirteen powerups range from a speed burrito to an uzi to a cork that plugs you up for four seconds. The Green Burrito sprays poo that piles up, sticks to bodies and hats, and stays on the level for the rest of the run.

The main mode is High Score: single-screen levels in order, three lives, one score for the whole run, a bonus round every three levels, and no retry. A run ends on a GAME OVER card with a roast line and superlatives, and every run can be saved as a replay and exported as a 25- to 60-second vertical clip or a 16:9 widescreen video. Three more modes share the same characters and rules: Burrito Arena (casual deathmatch or co-op), Survival (60 waves) and Gauntlet (versus AI, difficulty rising 10% per wave with no cap).

99 hand-built levels across six terrain themes exist today and ship as the free demo on itch.io and Steam. The full game is 200 levels plus an endless mode whose difficulty keeps climbing. The game is playable now in all four modes with local co-op and versus, replays, clip export, Steam leaderboards and about 418 logged development hours. The capstone work for this course is the content that gets the game from 99 levels to 200 and endless, plus the new playable characters feedback has asked for, produced by a crew of AI agents that work at development time, not at runtime. Nothing an agent makes reaches a player until it has been built, tested and played by a human.

- **Pillar 1.** Stomp or be stomped. Height is power; flap timing and position decide every fight.
- **Pillar 2.** Couch chaos. Two players on one screen, as partners or rivals, in every mode.
- **Pillar 3.** Gross-out humor that leaves a mark. Poo is physical and the base coat never washes off.
- **Pillar 4.** Every run is a clip. Every run is recorded 1:1 and can be shared as a vertical or widescreen video.

Target players: people who like short-session arcade action and chasing a high score, friends and families who want a couch game, fans of crude humor, and short-form video viewers.

1.1 Release intent: itch.io first, then Steam

Bottom Busters ships on two stores on purpose, and each store has a different job. itch.io is the proving ground. The 99-level demo goes up on itch.io first, as this course's release target, as a Draft or password-restricted page for classmates and a small group of testers, and then public. itch.io is where the difficulty curve, run length, the end card and the agent-made content get played by strangers before anything is promised on a store page that sells. Builds push through itch's butler tool, so an update is one command and testers get it the same day.

Steam is the commercial release. The same 99-level demo lists on Steam in November 2026 to collect wishlists, enters Steam Next Fest (February 22 to March 1, 2027) and the Couch Co-Op Fest (February 8 to 15, 2027), and the full 200-level game with the endless mode launches in March or April 2027 at a proposed $9.99 on both stores. Steam brings the features that need an account behind them: global and friends leaderboards with watchable top-3 replays, Remote Play Together for online couch co-op, and Workshop-hosted replays. Steamworks approval for Tingle Games LLC is in its identity-verification step now.

Both stores are built from one Unity project with two build targets, never two copies. The itch build turns off the Steam define and leaves the two Steam DLLs out, so it runs with no Steam client and no App ID; local leaderboards, replays and clip export work the same. Every feature and every level the agent crew produces lands in both builds automatically, so the itch testers and the Steam buyers are always playing the same game.

The demo is free on both stores; itch stays at price parity with Steam once the full game sells, since Steam's terms require it.

2. Game Mechanics

2.1 The core loop

One level is one screen, about 18 by 10 units, with a ground strip along the bottom, water gaps cut into it, and short platforms at three heights. The player sees the whole level at once. The loop inside a level is:

- **1.** Spawn at your spawn point and flap for height. Flight is the only way up; there is no jump.
- **2.** Get above an enemy and drop onto it to pop a cheek. Land so both cheeks pop at once for a Stomp Kill (500 points) instead of a Cheek Pop (100). Chained kills in the air multiply x2, x3, x4.
- **3.** Dodge enemies that are above you. They play by the same rules, so a hit from above costs you a cheek.
- **4.** Grab powerups that float across the level (50 points each) to change the fight: speed, a weapon, a shield, a trap for the other player.
- **5.** Clear every enemy on the screen and the next level loads. Score and lives carry across the whole run.

The loop across a run: levels climb through six themed worlds, a bonus round comes every three levels (catch food flipped by chef animals, 500 points each, nobody can die), and the run ends when the last life is gone. The GAME OVER card shows the total, the best, a roast line and superlatives, then offers Save Replay. In the demo the run also ends, as a win, on the end card after Bonus Round 33, which follows level 99.

2.2 Win and loss conditions

Mode	You win when	You lose when
High Score (1P or 2P co-op)	You clear the last level and its bonus round (level 99 in the demo, 200 in the full game). Otherwise the goal is the highest run score on the leaderboard.	All lives are gone. No retry; a run is a run.
High Score versus	You end the run with the higher score.	You end it with the lower score, or lose all lives first.
Survival (1P or 2P co-op)	You clear wave 60 (VICTORY).	All lives are gone.
Gauntlet (1v1 or 2v2 vs AI)	There is no end; two boards rank your highest wave and your most points.	Your side is eliminated in a wave.
Burrito Arena versus	You are the last player standing (three lives each).	You lose all three lives.
Endless (planned)	No end; you are ranked by the level reached.	All lives are gone.

2.3 Rules that keep the loop consistent

- **Symmetry.** Everyone has two cheeks, including enemies, so the same hit rules apply to both sides. One difficulty number controls how aggressive an enemy is: 1.0 to 1.3 in High Score levels, 1.4 to 2.2 by tier in Survival, and +0.1 per wave in Gauntlet.
- **Screen wrap.** Fly off the left edge and you come in on the right. Edge copies of a character can be hit and the hit counts.
- **Instant death.** Water and lava kill when your center is in them, never on a graze. The splat bat and bomb pickups are instant deaths too. The gum bubble absorbs one pop but not water or bombs.
- **Lives.** Three lives per player, uncapped. A 1UP adds one or revives a knocked-out co-op partner. After a death you respawn and flicker invincible for one second.
- **Scoring is earned.** Kills by bombs, whoopee cushions, the bat, drowning or a dead player's flying body score for nobody, so the only way to score is to stomp, shoot or burn an enemy yourself.

- **Levels and replays.** Levels are static data, built once and shipped as-is. Replays are recordings, not re-simulations, so a balance patch never breaks a saved replay.

2.4 Powerups (player-facing)

Burrito: 2.5x speed for 5 s. Green Burrito: uncontrollable flight while spraying poo. Ham: restores a cheek. Gum: a bubble that absorbs one pop. 1UP: one life. Uzi: 3-second burst, one bullet pops one cheek, streaks call out DOUBLE through MEGA KILL. Chili: every flap shoots a flame for 6 s. Cork: no flapping for 4 s, then a launch. Whoopee Cushion: planted on the ground; whoever lands on it loses a cheek and shoots into the ceiling. Spinning Platform: a rotating hazard. Splat Bat and Bomb: traps that kill whoever grabs them. In the arena modes, enough poo on the ground summons a toilet that falls on the biggest pile and flushes it.

3. AI Architecture

Two kinds of AI exist in this project and they never mix. In the shipped game, the only AI is the enemy state machine (Flying, Diving, Stunned, Idle) driven by that one difficulty number, plus pre-recorded announcer voices and a pre-written roast line bank. No large language model runs on a player's machine or is called while the game runs. The agent crew below works at development time on the project's own files, and its output is ordinary game data: level files, tuning tables, text, sprites and test reports. The game already discovers levels, skins and AI variants at runtime from data, so agent output drops in without code changes.

Each agent has one job and one output, and no two agents write the same file. The player never sees an agent; the player sees what the agent made.

Agent	Development role (one sentence)	What the player sees because of it
Level Architect	Writes new single-screen level files (ground gaps, platforms, spawn points, ceiling-drop markers, theme) that obey the one-screen rules in section 2.	Levels 100 to 200 and the endless mode's layouts: fresh platform arrangements and water gaps instead of the same three layouts repeating.
Difficulty Tuner	Reads the per-run event log from playtests and proposes enemy count and the difficulty number for each level and each endless-mode step.	A curve that actually climbs after level 8, and an endless mode where enemies dive sooner and track better every few levels until the run ends.
Roast Writer	Rewrites the roast line bank and superlatives, tagged to the run events that trigger them (died dirty, held it in, drowned, flawless).	A GAME OVER card that roasts what you actually did this run, from text shipped inside the game.
Wardrobe Stylist	Produces new hat and shades sprites and backdrop picks that pass the house sprite standard (size, density, point filtering, color regions).	More hats and shades on the Character Select screen, all of which still get dirty from poo like the originals.
Character Artist	Makes new playable characters: the body sprite, its 12 death-flail frames and a Character Select tile, to the house sprite standard, so hats, shades, cheek sets and poo stains sit on it like the original ten.	More characters to pick on the Character Select screen, which feedback on the build asked for. They drop in through the game's runtime skin discovery.
QA Playtester	Loads every new level in a headless Unity build and checks that each enemy is reachable, no spawn is over water, and the level can be cleared, then files a pass or fail report.	No unwinnable or empty level ever reaches the demo; a failed level goes back to the Level Architect, not to the player.
Builder	Reads this GDD and the project's coding rules and implements one approved feature at a time in Unity C#, then runs the existing build-and-verify loop.	The endless mode itself, the Custom Game screen behind the current menu teaser, and whatever the other agents' output needs to load.

How they fit together: the Level Architect makes a level, the QA Playtester passes or fails it, the Difficulty Tuner sets its numbers after real playtest data exists, and the Builder only changes code. The Character Artist and the Wardrobe Stylist both make sprites but never the same files: characters on one side, hats and shades on the other. The handoff between agents is a file in the repository, not a conversation, so any step can be re-run alone. A human (me) is the last gate: I play every build and nothing is committed without my say-so. That rule already exists in the project and the agents inherit it.

4. Technical Strategy

4.1 Agent roles and boundaries

Agent	Reads	Writes	Never touches
Level Architect	Level rules, theme list, existing level files	New level files	Code, tuning, art
Difficulty Tuner	Playtest event logs, current tuning table	Tuning table (enemy count, difficulty per level)	Level layouts, code
Roast Writer	Scoring and event categories, current line bank	Roast and superlative text files	Anything but text
Wardrobe Stylist	Sprite standard, existing hat sprites	Hat and shades sprites plus import settings	Levels, code
Character Artist	Sprite standard, the ten existing skins and their flail frames	New skin sprites, flail frames and select tiles plus import settings	Hats and shades, levels, code
QA Playtester	New level files, headless build	Pass/fail report per level	Any game file
Builder	This GDD, CLAUDE.md, the C# source	C# source, scenes	Level data, tuning, text, art

4.2 Token budget per agent (semester)

Each figure is tokens in plus tokens out across all calls, with a retry allowance built in. The Builder dominates because every call carries the GDD, the coding rules and the relevant source files as context (about 40,000 tokens per call). The content agents are cheap because a level file is a few hundred tokens and a roast line is a sentence.

Agent	Unit of work	Tokens per unit	Units	Budget
Level Architect	One level, with one retry on QA fail	6,000	101 levels + 40 endless templates	850,000
Difficulty Tuner	One tuning pass over all levels	25,000	10 passes (one per playtest round)	250,000
Roast Writer	One batch of 25 lines	8,000	10 batches (250 lines)	80,000
Wardrobe Stylist	One hat or shades set	5,000	40 items	200,000
Character Artist	One playable character (body, 12 flail frames, tile), one redo	15,000	10 characters	150,000
QA Playtester	One level check and report	2,500	180 checks (levels plus re-checks)	450,000
Builder	One feature slice, build and verify	500,000	6 slices (endless mode, custom game, loaders)	3,000,000

Total				about 5.0 million

At current Claude Sonnet list prices this is tens of dollars for the semester, not hundreds; the exact figure is checked against the pricing page before each phase. The budget is a cap, not a target, and the Builder's share is the one to watch: if a slice runs past 500,000 tokens the slice is too big and gets split.

4.3 Constraints

- **No runtime API calls.** The game must cost $0 per player to run, work offline, and stay inside Steam's pre-generated-content disclosure. So every agent runs at development time and ships only static data. This rules out live roast generation, live level generation and any API key inside the build. It is the single biggest constraint and it is deliberate.
- **Build time, not tokens, is the QA bottleneck.** The QA Playtester is bound by Unity's headless build time, roughly three to six minutes per build on the development PC, not by tokens. Levels are checked in batches per build, and a batch is capped at 20 levels so one bad level does not cost a 40-minute rerun.
- **Context size.** The Builder's context is bounded by the files it needs; the project's coding rules already live in one file (CLAUDE.md) and the GDD is this document, so a Builder call stays near 40,000 input tokens and never reads the whole 191-file source tree.
- **Scope.** The endless mode is the one new system; everything else is content for systems that already run. If the endless mode slips, the 200-level game still ships and the agents' level, tuning and text output is still used.
- **Human gate.** Nothing an agent makes is committed, built for a store, or shown to players without a human playtest. This is the existing project rule and the pipeline is built around it.